# DP-OPS-0010: ARCHITECTURAL TRANSFORMATION FOR COGNITIVE LIVING SURFACES AND SYSTEM LEARNING

## 1. Introduction: The Evolution of Repository Governance

The Stela platform stands at a critical inflection point in its architectural history. Originally conceived as a fork of the "PHP-Nuke CE" Content Management System, the repository has evolved far beyond its initial payload to become a "Governance Operating System"—a rigorous, text-based environment where the management of code is treated with higher precedence than the code itself.[1] This evolution has been driven by the implementation of "Dispatch Packets" (DPs), strict jurisdictional boundaries between Human Operators and AI Agents (Integrators/Contractors), and a "Filing Doctrine" that prioritizes explainability over convenience. However, the current governance model, while robust for maintaining state, lacks a native mechanism for *learning* and *capability persistence*.

The existing "Living Surface," TASK.md, serves as an excellent ephemeral workspace for active threads, but its contents are destined for the archives or the trash once a task is complete. The "State of Play" (SoP.md) records the history of *what* happened, but not the detailed procedural knowledge of *how* complex operations were executed. This creates a "Knowledge Gap" where the system must re-discover optimal patterns for every new task, leading to efficiency drift and potential inconsistencies in execution. To bridge this gap, the system requires a new class of artifacts: "Root Living Surfaces" that persist capabilities (SKILL.md) and high-level objectives (PROJECT.md) across the operational lifespan of the repository.

This report details the architectural design and implementation plan for **DP-OPS-0010**, a transformative Dispatch Packet tasked with establishing these new surfaces, deploying the S-LEARN tooling to operationalize knowledge extraction, and formally decoupling the platform from its legacy PHP identity. By executing this plan, Stela transitions from a static file repository into a "Cognitive Architecture" capable of accumulating operational wisdom, thereby reducing the cognitive load on both human operators and AI agents in future cycles.

---

## 2. Deconstructing the Stela Identity: The Decoupling

# of Legacy PHP

Before new capabilities can be grafted onto the system, the foundational identity of the platform must be clarified. The current repository metadata presents a conflict between its actual function and its stated identity. The "PHP-Nuke CE" branding acts as a conceptual anchor, dragging the governance model back toward a specific runtime environment that no longer defines the platform's core value proposition.

## 2.1 The Semiotics of the README and Configuration

An audit of the repository's root artifacts reveals specific "PHP" markers that are now misaligned with the "Explainability-First" ethos defined in TRUTH.md.[1] The README.md file explicitly titles the project "Stela (PHP-Nuke CE)" and describes it as a "CMS program".[1] This description fails to capture the reality of the ops/, docs/, and tools/ directories, which constitute a language-agnostic governance layer built on Bash and Markdown.

Furthermore, the .editorconfig file contains a dedicated block for [*.php], enforcing indentation rules for a language that may not even exist in future projects managed by this platform. The SECURITY.md file references "Legacy code paths include MD5 hashing," a vestige of the PHP-Nuke heritage.[1] While accurate for the legacy payload, strictly binding the security policy of the *platform* to the technical debt of a *specific project* violates the separation of concerns mandated by the Stela Filing Doctrine.

## 2.2 The Strategic Imperative for Agnosticism

Removing these PHP references is not merely a "cleanup" task; it is a constitutional amendment. By stripping the "PHP-Nuke" label, we redefine Stela as a "Universal Governance Shell." This allows the projects/ directory to host any technology stack—Rust, Node.js, Python, or Go—while retaining the same rigorous governance model.

The removal of PHP references serves three strategic goals:

1. **Jurisdictional Clarity:** It clarifies that the *Platform* (ops/docs/tools) is distinct from the *Payload* (projects/). The Platform is the Governor; the Payload is the Governed.
2. **Cognitive Unlocking:** It signals to the AI Integrator that it should not assume PHP constraints when drafting DPs for new projects.
3. **Future-Proofing:** It prepares the repository for multi-project tenancy, where different projects may use conflicting technology stacks, managed by a unified PROJECT.md registry.

Consequently, DP-OPS-0010 must execute a surgical removal of these references. The README.md will be rebranded to "Stela Platform." The .editorconfig will be generalized. The SECURITY.md will be reframed to address the security of the *governance layer* and generic payload risks, moving specific PHP vulnerability details to the project-level documentation

where they belong.

---

# 3. Architectural Design: The Trinity of Living Surfaces

The core innovation of DP-OPS-0010 is the introduction of SKILL.md and PROJECT.md. To understand their necessity, we must analyze the "Stateless" nature of the Stela repository. As defined in docs/CONTEXT.md, "Context is not memory; it is a file".[1] When an AI agent is instantiated via ops/bin/open, it has no memory of previous sessions other than what is loaded into its context window.

Currently, the context load includes:

- **Constitution:** TRUTH.md (Laws)
- **History:** SoP.md (Events)
- **Active Task:** TASK.md (Current Objective)

This triad lacks a "Capability" dimension. If an agent learned how to perfectly format a "Risk Assessment Table" in Session A, that knowledge is lost in Session B unless the Operator explicitly manually pastes it. SKILL.md fills this void.

## 3.1 SKILL.md: The Cognitive Template

SKILL.md is designed to be the "Template at the Root." It serves as the canonical repository for "Learned Operational Patterns." Unlike SoP.md, which is a log of events, SKILL.md is a library of *methods*.

### 3.1.1 The Inheritance Model

The requirement for SKILL.md to act as a "template" implies an inheritance model. The root file defines the *schema* for a skill, and potentially hosts the "Core Skills" required for basic repository operations. Project-specific skills might inherit from this structure in the future, but for now, the root SKILL.md serves as the centralized "Brain" of the repository.

### 3.1.2 Detailed Schema Design

The efficacy of SKILL.md relies entirely on its machine-readability. It must be structured so that an AI Integrator can parse it deterministically. We propose the following schema for the file:

**Header Section:**

- **Identity:** SKILL.md
- **Status:** Living Surface
- **Owner:** Integrator (The AI Lead)

- **Purpose:** To persist high-value operational patterns and reduce re-discovery costs.

**Section 1: The Skill Registry:**

A high-density table mapping Skill IDs to their utility. This allows an agent to quickly scan available capabilities without reading the full text of every skill (token optimization).

**Section 2: The Skill Template (The Schema):**

This section explicitly defines *how* to write a new skill. This is crucial for the S-LEARN tooling, which needs a target format. The schema includes:

- **Trigger:** The condition under which this skill should be activated.
- **Context Cost:** An estimate of how many tokens or files are needed to execute it.
- **Prerequisites:** Tools or permissions required.
- **Procedure:** A step-by-step algorithmic execution path.
- **Validation:** How to prove the skill was executed correctly.

**Section 3: Core Skills:**

The initial population of the file. This might include skills like "Drafting a Standard DP" or "Performing a Context Lint."

## 3.2 PROJECT.md: The Objective Anchor

While SKILL.md handles the "How," PROJECT.md handles the "Why" and "Where." Currently, the projects/ directory is an unstructured bucket. PROJECT.md introduces a "Federated" governance model.

### 3.2.1 The Concept of the "Active Project"

In a multi-project repository, the TASK.md might be focused on one specific sub-project. PROJECT.md serves as the "Meta-Contract" for that specific domain. It defines the "Rules of Engagement" for a specific codebase (e.g., "This project uses Rust and enforces cargo fmt").

### 3.2.2 The Registry Pattern

The user request includes a "Projects Registry." This registry serves as the lookup table between a human-readable "Slug" (e.g., cms-core) and the filesystem path (e.g., projects/php-nuke-ce/). This registry is best placed in ops/lib/project/REGISTRY.md, separating the *data* (the list of projects) from the *interface* (PROJECT.md).

The interaction model is as follows:

1. **Operator** initiates work on "Project X".
2. **Tooling** checks ops/lib/project/REGISTRY.md to find the path.
3. **Tooling** loads projects/X/PROJECT.md (if it exists) or falls back to the root PROJECT.md

as the default template.

This structure allows the root PROJECT.md to act as the "Default Project Configuration," ensuring that even new, undefined projects have a baseline governance structure.

### 3.3 Integration with the Context Manifest

These new surfaces are useless if the AI agents are not aware of them. The ops/lib/manifests/CONTEXT.md file is the "Gatekeeper of Awareness".[1] It dictates what files are loaded into the AI's context window.

DP-OPS-0010 must update ops/lib/manifests/CONTEXT.md to include:

- SKILL.md (Mandatory Core Context)
- PROJECT.md (Mandatory Core Context)
- ops/lib/project/REGISTRY.md (Supporting Surface)

By adding these to the manifest, we ensure that every time ops/bin/open is run, the agent is immediately "rehydrated" with the system's capabilities and project definitions. This creates the "Living" aspect of the surface—it is always present, always active.

---

## 4. Operationalizing Wisdom: The S-LEARN Tooling

The most dynamic component of DP-OPS-0010 is the creation of tools/S-LEARN-01. The naming convention S-LEARN suggests "System Learning." The 01 suffix implies this is the first of a generation of learning tools, or perhaps a specific "Lesson 1" script. Given the context of establishing a governance loop, we interpret S-LEARN-01 as the "Skill Extraction Utility."

### 4.1 The Theory of System Learning

In a standard DevOps cycle, "Lessons Learned" are often relegated to a post-mortem meeting and then forgotten. In Stela, we aim to operationalize this. The S-LEARN loop follows this logic:

1. **Execution:** A DP is executed and verifiable results are produced in storage/handoff/.
2. **Reflection:** The S-LEARN script is triggered. It analyzes the results.
3. **Extraction:** The Operator (or Agent) identifies a repeatable pattern from the execution.
4. **Crystallization:** This pattern is formatted according to the SKILL.md template and appended to the file.
5. **Availability:** The new skill is now available for all future sessions via Context Loading.

### 4.2 Script Design: tools/S-LEARN-01

The script must be a robust Bash executable, consistent with the existing tooling suite (tools/dp_lint.sh, ops/bin/dump).[1] It needs to perform the following actions:

1. **Locate Context:** Find the repository root and verify the existence of SKILL.md and storage/handoff/.
2. **Identify Artifacts:** Scan storage/handoff/ for the most recent RESULTS.md file.
3. **Interactive Dialogue:** Prompt the user to confirm if the recent session yielded a reusable skill.
4. **Template Generation:** If affirmative, generating a "Skeleton Skill" based on the metadata of the recent DP (ID, Title, Objective).
5. **Append/Edit:** Allow the user to edit this skeleton and append it to SKILL.md.

#### 4.2.1 Detailed Logic Analysis

The script requires set -euo pipefail for safety. It should use grep or awk to parse the RESULTS.md file to extract the "Scope summary" and "Verification" steps, as these often form the basis of a skill's "Trigger" and "Validation" sections.

The script serves as a "forcing function." By existing as a tool that can be run (and potentially mandated in future TASK.md closeout procedures), it forces the habit of reflection. It prevents the system from becoming a "mindless executor" and pushes it toward being a "learning optimizer."

---

## 5. Detailed Implementation Plan: DP-OPS-0010

The following section presents the definitive text for **DP-OPS-0010**, drafted to meet the exact specifications of the TASK.md template found in the research dump.[1] This packet is the instrument of change.

### 5.1 DP-OPS-0010 Specification

# DP-OPS-0010: Establish Root Living Surfaces & S-LEARN Tooling

## 0. FRESHNESS GATE (MUST PASS BEFORE WORK)

- **Base Branch:** main
- **Required Work Branch:** work/ops-living-surfaces-2026-01-31
- **Base HEAD:** [Current Hash]

## I) REQUIRED CONTEXT LOAD (READ BEFORE DOING ANYTHING)

- **Loaded:** OPEN, TRUTH, AGENTS, SoP, CONTEXT, MAP
- **Job-specific canon:**
  - ops/lib/manifests/CONTEXT.md
    -.editorconfig
  - README.md
  - SECURITY.md
  - tools/context_lint.sh (for verification reference)

# II) SCOPE & SAFETY

- **Objective:** Establish SKILL.md and PROJECT.md as root living surfaces to persist capabilities and project definitions, deploy the S-LEARN learning tool, and formally decouple the platform from legacy PHP references to ensure true governance agnosticism.
- **Non-Goals:** This DP does not involve refactoring the legacy CMS code itself, only the governance metadata and root configuration files.
- **Target Files allowlist (hard gate):**
  - SKILL.md (new)
  - PROJECT.md (new)
  - tools/S-LEARN-01 (new)
  - ops/lib/project/REGISTRY.md (new)
  - ops/lib/manifests/CONTEXT.md
    -.editorconfig
  - README.md
  - SECURITY.md
  - SoP.md

# III. EXECUTION PLAN (A–E CANON)

## A) STATE

- **Context:** The repository currently relies on TASK.md for active intent and SoP.md for history. There is no persistent memory for "Skills" (how-to knowledge) or "Projects" (domain definitions). The repository also carries "PHP-Nuke CE" branding that contradicts its platform-agnostic governance capability.
- **Drift:** The lack of SKILL.md forces agents to "re-invent" workflows every session. The presence of [*.php] in .editorconfig and "CMS program" in README.md creates a false constraint on the Integrator's architectural decisions.
- **Desired State:** SKILL.md and PROJECT.md exist at the root. S-LEARN tooling is available to harvest skills. The Context Manifest includes these new surfaces. All PHP-specific references are removed from root files.

## B) REQUEST

1. **Create SKILL.md (Root Living Surface):**
    - Act as the **Template at the Root**.
    - Include a "Skill Registry" table.
    - Define the "Skill Schema" (Trigger, Prerequisites, Execution, Validation).
    - Initialize with a philosophy section explaining that Skills are "Crystallized Logic."
2. **Create PROJECT.md (Root Living Surface):**
    - Define the "Project Contract" (Slug, Stack, Scope, Roadmap).
    - Serve as the default template for any project lacking its own definition.
3. **Create ops/lib/project/REGISTRY.md:**
    - Establish a markdown table mapping Project Slug -> Filesystem Path.
    - Initialize with the current project (if applicable) or leave as a template structure.
4. **Create tools/S-LEARN-01 (Script):**
    - Language: Bash (#!/usr/bin/env bash).
    - Permissions: Executable (chmod +x).
    - Logic: Scan storage/handoff/, find the latest result, prompting the user to extract a skill, and appending a template to SKILL.md.
5. **Update ops/lib/manifests/CONTEXT.md:**
    - Add SKILL.md to the "Canonical Context" list.
    - Add PROJECT.md to the "Canonical Context" list.
    - Add ops/lib/project/REGISTRY.md to "Supporting Surfaces."
    - **Constraint:** Do not rename the file.
6. **Remove PHP References (Decoupling):**
    - **README.md:** Change title from "Stela (PHP-Nuke CE)" to "Stela Platform". Remove "CMS program".
    - **.editorconfig:** Delete the [*.php] indentation block entirely.
    - **SECURITY.md:** Remove specific references to "Legacy code paths include MD5 hashing" if they are framed as PHP-specific issues, or generalize them to "Legacy Payload Risks." Ensure the text reflects a governance platform, not a PHP script.

## C) CHANGELOG

- Added SKILL.md as the root template for system capabilities.
- Added PROJECT.md as the root template for project definitions.
- Added tools/S-LEARN-01 to operationalize skill extraction.
- Added ops/lib/project/REGISTRY.md to track project paths.
- Updated ops/lib/manifests/CONTEXT.md to enforce loading of new living surfaces.
- Removed legacy PHP branding and configuration to establish platform agnosticism.

## D) PATCH / DIFF

(Worker to generate unified diffs for all targeted files).

## E) RECEIPT (REQUIRED)

### Verification

- **Tree Check:** bash tools/verify_tree.sh (Must Pass).
- **Context Lint:** bash tools/context_lint.sh (Must Pass; verify it detects SKILL.md and PROJECT.md).
- **Truth Lint:** bash tools/lint_truth.sh (Must Pass).
- **PHP Audit:** grep -r "PHP" README.md.editorconfig SECURITY.md (Expect 0 matches).
- **Tool Exec:** bash tools/S-LEARN-01 (Run in dry-mode or test to ensure no syntax errors).

---

# 6. Implementation Mechanics: The S-LEARN Script

To satisfy the requirement for exhaustive detail, we must look at the specific implementation of tools/S-LEARN-01. This script is not a trivial utility; it is the "Synapse" of the new cognitive architecture.

The script must handle the reality of the Stela filesystem. It must locate the handoff directory, robustly handle filenames that include timestamps and hashes (e.g., OPEN-main-e29ab6b23.txt), and parse the structured text of RESULTS.md.

**Proposed Logic Flow for S-LEARN-01:**

1. **Shebang & Safety:** Standard Bash hygiene.
2. **Path Resolution:** Calculate REPO_ROOT dynamically to ensure it runs from anywhere.
3. **Discovery:** Use find or ls -t to locate the most recent file matching *RESULTS.md in storage/handoff/.
4. **Parsing:** Read the file. Extract the line starting with Scope summary: to use as the default description for the new skill.
5. **Interaction:** Output "Found latest session: [Filename]. Extract skill? [y/N]".
6. **Template Injection:** If yes, append a formatted block to SKILL.md.

**The "Template" to be Appended:**

### SKILL-:

**Origin:** Derived from session

**Context:**

**Logic:**

1.

2.

This automated linkage creates a "Provenance Chain" from the Skill back to the specific work

session where it was learned. This is crucial for the "Traceability" requirement mentioned in CONTRIBUTING.md.[1]

---

# 7. Governance Risks and Mitigation

The introduction of Root Living Surfaces introduces specific governance risks that must be managed.

## 7.1 The "Context Bloat" Risk

By mandating SKILL.md and PROJECT.md in the CONTEXT.md manifest, we are permanently increasing the "Input Token Cost" of every single AI interaction. If SKILL.md grows to 50 pages, it will crowd out the TRUTH.md constitution in the AI's context window.

**Mitigation:** SKILL.md must be curated. It is not a log; it is a *Registry*. The "Skill Definitions" might eventually need to be moved to a docs/skills/ directory, with SKILL.md acting only as the index. However, for DP-OPS-0010, keeping it as a single root file is the correct "MVP" (Minimum Viable Product) approach to establish the pattern.

## 7.2 The "Merge Conflict" Risk

If multiple branches are working on different DPs, and all of them attempt to update SKILL.md with new learnings, merge conflicts on main will be frequent.

**Mitigation:** This reinforces the AGENTS.md rule that the "Integrator" owns the structure. Updates to SKILL.md should perhaps be batched or handled in specific "Governance Cycles" rather than ad-hoc appended by every Contractor. The S-LEARN tool currently only *appends* locally; the Operator must consciously choose to commit that change. This "Human-in-the-Loop" for skill commitment is a critical safety valve.

---

# 8. Conclusion

DP-OPS-0010 represents the maturation of the Stela platform. By shedding the "PHP-Nuke" skin, it acknowledges its reality as a general-purpose governance engine. By establishing SKILL.md and PROJECT.md, it moves from a "Stateless" execution model to a "Stateful" cognitive model, capable of remembering capabilities and defining persistent objectives. The S-LEARN tooling provides the mechanical means to execute this cognitive cycle.

This architectural shift aligns perfectly with the "4-Line Heuristic" of the Stela Filing Doctrine:

- It improves the **RUN** capability (ops/bin/project, tools/S-LEARN-01).
- It enhances the **EXPLAIN** capability (SKILL.md as operational documentation).

- It clarifies the **WORK** structure (PROJECT.md and Registry).

The execution of this DP is not just a feature addition; it is a refounding of the platform's operational philosophy.

---

# 9. Appendix: Artifact Reference Data

## Table 1: New Artifact Definitions

| Artifact | Location | Type | Owner | Purpose |
|---|---|---|---|---|
| **SKILL.md** | /SKILL.md | Living Surface | Integrator | Stores reusable operational patterns and capability definitions. |
| **PROJECT.md** | /PROJECT.md | Living Surface | Operator | Defines the active project's scope, stack, and roadmap. |
| **Registry** | ops/lib/project/REGISTRY.md | Library Data | Integrator | Maps Project Slugs to Filesystem Paths. |
| **S-LEARN** | tools/S-LEARN-01 | Script (Bash) | Integrator | Interactive utility to extract skills from DP results. |

## Table 2: PHP Reference Audit & Removal Plan

| File | Current Content | Action | New Content |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **README.md** | "Stela (PHP-Nuke CE)" | Rename | "Stela Platform" |
| **README.md** | "CMS program" | Generalize | "Governance System" |
| **.editorconfig** | [*.php] section | Delete | (None) |
| **SECURITY.md** | "Legacy code paths include MD5..." | Reframing | "Legacy payloads may contain weak hashing..." |

### Table 3: Context Manifest Updates (ops/lib/manifests/CONTEXT.md)

| Section | Action | Entry |
|---|---|---|
| **Canonical Context** | Add | SKILL.md |
| **Canonical Context** | Add | PROJECT.md |
| **Supporting Surfaces** | Add | ops/lib/project/REGISTRY.md |

This concludes the architectural report for DP-OPS-0010. The system is now ready for the Dispatch Packet to be formally drafted and executed by the Contractor.

## 10. Extended Analysis: The "Living Surface" Theory

To fully justify the architectural shift required by DP-OPS-0010, one must delve deeper into the theoretical underpinnings of "Living Surfaces" within the context of AI-Human collaboration. The traditional "Static File" model of software repositories—where files like README.md or CONTRIBUTING.md are updated only rarely—fails to capture the high-velocity state changes inherent in an AI-driven workflow.

### 10.1 The Static vs. Dynamic Dichotomy

In the current Stela architecture, TRUTH.md represents the "Static" pole—immutable laws that rarely change. TASK.md represents the "Dynamic" pole—a file that changes hourly as the Work Log is updated. The gap lies in the middle ground: "Meta-State."

Meta-State refers to information that is too fluid to be "Law" (Constitution) but too permanent to be "Task" (Ticket).

- **Skills:** A skill like "How to run the linter" is not a Law (it might change if the tool changes), but it is not a Task (it is reusable).
- **Projects:** A project definition (e.g., "We are building a React App") is not a Law (we might switch to Vue), but it is not a Task (it persists for months).

SKILL.md and PROJECT.md are the containers for this Meta-State. By placing them at the **Root**, we signal their importance to the repository's hierarchy. They are peers to the Constitution, not subordinates to the Documentation.

### 10.2 The Cognitive Load Argument

For a Human Operator, "Context" is implicit. We remember what we did yesterday. For an AI Agent (Contractor), Context is explicit and expensive. Every time an agent is spun up, it is a "tabula rasa" (blank slate). It only knows what is in the file dump.

Without SKILL.md, the AI must "hallucinate" or "re-deduce" the correct way to perform an operation based on scattered clues in the docs/ folder. This burns tokens and introduces variance.

- **Without SKILL.md:** Agent reads 5 docs -> Deduce procedure -> Execute (High Variance).
- **With SKILL.md:** Agent reads 1 Skill -> Execute (Low Variance).

Therefore, DP-OPS-0010 is largely a "Token Optimization" and "Reliability Engineering" effort. It compresses scattered documentation into crystallized, executable patterns.

## 11. The Psychology of S-LEARN

The naming of tools/S-LEARN-01 invites a psychological analysis of the system's design. Why "S-LEARN"? The prefix "S" likely stands for "Stela" or "System." The command acts as a "Cognitive Checkpoint."

### 11.1 The "Lesson" Metaphor

By numbering it 01, we imply a curriculum. The system is "in school." The Operator is the Teacher; the Repository is the Student.

- **Lesson 01:** Extraction. The ability to look at what you just did and summarize it.

This mirrors the "Reinforcement Learning from Human Feedback" (RLHF) loop, but implemented via filesystem artifacts rather than neural weights. The SKILL.md file becomes the "Weights" of the repository-level brain.

### 11.2 Tooling as Ritual

In strict governance systems, behavior is shaped by tools. If there is no tool to extract a skill, no skills will be extracted. By creating a script that *prompts* the user ("Does this session represent a reusable pattern?"), we inject a "Ritual of Reflection" into the closeout process. This ritual is essential for preventing the entropy described in docs/GOVERNANCE.md as "Drift".[1]

## 12. Future Roadmap: DP-OPS-0011 and Beyond

DP-OPS-0010 is a foundational packet. It builds the shelves (SKILL.md, PROJECT.md) but does not fully stock them.

### 12.1 Immediate Follow-on (DP-OPS-0011)

Once the surfaces exist, the next logical step is population.

- **Intent:** "Populate Core Skills."
- **Action:** Run S-LEARN on the last 10 historical DPs (from SoP.md history) to backfill SKILL.md with proven patterns.

### 12.2 The "Auto-Pilot" Vision

Eventually, the Integrator agent could be granted the authority to run S-LEARN autonomously.

- **Scenario:** Integrator detects a successful verify_tree run. Integrator parses the steps used. Integrator auto-proposes a PR to update SKILL.md with a "Tree Verification Skill."

This leads to a self-healing, self-documenting repository. DP-OPS-0010 is the prerequisite for this autonomous future.

## 13. Final Governance Check

Before concluding, we must verify that DP-OPS-0010 itself adheres to the TRUTH.md invariants.

- **Does it RUN?** Yes, via tools/S-LEARN-01.
- **Does it EXPLAIN?** Yes, via SKILL.md.
- **Is it WORK?** Yes, via PROJECT.md.
- **Is it TRASH?** No.

The packet respects the "4-Line Heuristic." It also respects the "Jurisdictional Trinity" by assigning clear ownership (Integrator vs Operator) to the new surfaces. The removal of PHP references respects the "Drift" invariant by aligning the repository's description with its actual state.

The system is green for launch.

**(End of Report)**

## Works cited

1. dump-full-main-e29ab6b23.txt